

MPI

MPI

MPI, *Message Passing Interface*, is a *standard* for writing parallel programs on **distributed** memory systems.

MPI is a *library* that provides functions for `C` and `Fortran` (and some other languages) to send *messages* between processes.

As well as some *global* messages between all processes called **collective** communications

We'll go over MPI, its functions, and some issues present when using a memory passing model

And finally revisit performance

Installation

MPI is available on most systems and it's primarily used for high-performance computing clusters.

We can install it on our local machines where each core is treated as a separate process and acts like a node in a cluster.

[mpich.org](https://www.mpich.org) is a popular implementation of MPI.

For windows, it leads you to `MS-MPI` which is a Microsoft implementation of MPI.

For apple, you can install `mpich` using `brew install mpich` or `sudo port install mpich`

This should mean you have

```
1 mpicc
2 mpirun
```

Available on your command line.

Hello world

Here is a simple "Hello World" program in MPI:

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <mpi.h>
4
5  const int MAX_STRING = 100;
6
7  int main(void) {
8      char greeting[MAX_STRING];
9      int comm_size;
10     int my_rank;
11
12     MPI_Init(NULL, NULL);
13     MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
14     MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
15
16     if (my_rank  $\neq$  0) {
17         sprintf(greeting, "from process %d of %d", my_rank, comm_size);
```

Compilation and execution

```
1 mpicc -Wall -o mpi_hello mpi_hello.c
```

`mpicc` is a *wrapper* for the `gcc` compiler that links the MPI library.

and you can also use `mpirun` or `mpiexec` to run the program with a specified number of processes:

```
1 mpirun -n 4 ./mpi_hello
```

Setup

```
1  #include <mpi.h>
```

Imports the MPI library and allows you to use MPI functions

Initialization

```
1 MPI_Init(NULL, NULL);
```

and

```
1 MPI_Finalize();
```

Determine which processes are participating in the MPI program and clean up resources when done.

No MPI functions should be called before the `MPI_Init` or after `MPI_Finalize`.

The specific syntax for `MPI_Init` can vary, but the most common form is

```
1 MPI_Init(int* argc_p, char*** argv_p);
```

Where `argc_p` and `argv_p` are pointers to the arguments passed in main. When a program has no arguments, we can simply pass `NULL` for both parameters.

Note that `MPI_Init` returns an `int` error code, we'll ignore it

Finalization

```
1 MPI_Finalize();
```

This states that we're done using MPI, and that *any resources allocated by MPI can be freed*.

```
1 int MPI_Finalize(void);
```

Communicators

In MPI, a *communicator* is a **collection of processes** that can send messages to each other

`MPI_Init` defines a communicator that consists of all the processes started by the user when they started the program

This is usually called `MPI_COMM_WORLD`, and we can get information out of this communicator using the following functions:

```
1 MPI_Comm_size(MPI_Comm comm, int* comm_size_p);  
2 MPI_Comm_rank(MPI_Comm comm, int* my_rank_p);
```

The first argument is the *communicator*, with a type defined by `MPI`,

And these functions return to a location, usually to an address defined in the *second argument*

SPMD

Notice that this is *one* program that is run by *multiple* processes.

Most MPI programs are written this way, and this is called **SPMD** (*Single Program, Multiple Data*).

Notice that our program can, in principle, run with any number of processes which is good practice

Communication

Communication

In our program, other than process 0, each process creates a message which will *send* to 0

And process 0 *receives* messages from all other processes.

It does this by *looping* over the number of processes, and calling `MPI_Recv` for each process.

MPI_Send

```
1  int MPI_Send(  
2      void* msg_buf_p,  
3      int msg_size,  
4      MPI_Datatype msg_type,  
5      int dest,  
6      int tag,  
7      MPI_Comm communicator  
8  )
```

The first three arguments, `msg_buf_p`, `msg_size`, and `msg_type` specify the message to be sent.

Specifically,

1. `msg_buf_p` is a pointer to the data to be sent,
2. `msg_size` is the number of elements in the message, for strings, that's for each character plus the null terminator, and
3. `msg_type` specifies the type of data that is defined by `MPI`.

MPI_Send

The next three arguments, `dest`, `tag`, and `communicator` specify where the message should be sent.

1. `dest` is the *rank* of the process that should receive the message,
2. `tag` is an integer that can be used to *label* the message, and
3. `communicator` specifies the communicator that the sender and receiver belong to.

You can think of `tag` as a way to *filter* messages.

For example, if process 0 is receiving messages from multiple processes, it can use the `tag` to determine which message to average and which message to print.

An `communicator` as a *world*, and you can have multiple communicators that represent different groups of processes. And they would have *no way* of communicating with each other.

MPI_Recv

```
1  int MPI_Recv(  
2      void* msg_buf_p,  
3      int msg_size,  
4      MPI_Datatype msg_type,  
5      int source,  
6      int tag,  
7      MPI_Comm communicator,  
8      MPI_Status* status  
9  )
```

The first three, `msg_buf_p`, `msg_size`, and `msg_type` specify where the received message should be stored.

- the `source` argument specifies the rank of the process that sent the message
- the `tag` argument specifies the tag of the message to be received
- the `communicator` argument specifies the communicator that the sender and receiver belong to
- the `status` argument is a pointer to an `MPI_Status` structure that can be used to get information about the received message, such as its size and source.

Message matching

Suppose process 1 uses `MPI_Send` with

```
1  MPI_Send(send_buf_p, msg_size, msg_type, dest, send_tag, communicator);
```

Then process 0 can receive this message using `MPI_Recv` with

```
1  MPI_Recv(recv_buf_p, msg_size, msg_type, source, recv_tag, communicator, &status);
```

The message sent by `1` will only be received by `0` if the following conditions are met:

1. `recv_communicator == send_communicator`
2. `recv_tag == send_tag`
3. `source == 1`
4. `dest == 0`

MPI_ANY_SOURCE

In `MPI_Recv`, we can use `MPI_ANY_SOURCE` as the `source` argument to receive a message from *any* source.

For example,

If `process 0` is sending work to processes `1, 2, ..., comm_sz-1`, and `process 1, 2, ..., comm_sz-1` send their results back to `process 0` when they finish

And if the time it takes to finish the work is *unpredictable*, then `process 0` has no way of knowing the order in which the results will be sent back.

In this case, `process 0` can use `MPI_ANY_SOURCE` to receive messages from any source, and it will receive the first message that is sent back.

It can also use `MPI_ANY_TAG` to receive messages with any tag, and it will receive the first message that is sent back regardless of its tag.

So instead of placing the sent messages into a buffer and waiting for a loop to receive them, we can use `MPI_ANY_SOURCE` and `MPI_ANY_TAG` to receive messages as they come in.

MPI `status_p` argument

A receiver can receive a message without knowing the amount of data in the message, the sender of the message, or the tag of the message

`MPI_Status` is a struct with at least 3 members

- `MPI_SOURCE` which gives the rank of the sender of the message
- `MPI_TAG` which gives the tag of the message
- `MPI_ERROR` which gives the error code of the message

TODO

Question

What happens in the greetings program if, instead of `strlen(greeting) + 1`, we use `strlen(greeting)`

What if we use `MAX_STRING` instead?